
UNIT 1 INTRODUCTION TO OBJECT ORIENTED MODELING

Structure

Page Nos.

- 1.0 Introduction
- 1.1 Objectives
- 1.2 Introduction to Object Orientation
- 1.3 Basic Philosophy of Object Orientation
- 1.4 Principles of Object Orientation
 - 1.4.1 Abstraction
 - 1.4.2 Encapsulation
 - 1.4.3 Inheritance
 - 1.4.4 Polymorphism
- 1.5 Basic Constructs in Object Orientation
 - 1.5.1 Class and Objects
 - 1.5.2 Links and Association
 - 1.5.3 Generalization and Inheritance
- 1.6 Identifying Classes
 - 1.6.1 An Object Model
- 1.7 Benefits of Object Orientation
- 1.8 Introduction to Object Oriented Analysis & Design Tools
- 1.9 Summary
- 1.10 Solutions/Answers to Check Your Progress
- 1.11 References and Further Readings

1.0 INTRODUCTION

Object Oriented Analysis and Design (OOAD) is an approach for developing software systems using object-oriented software engineering concepts. OOAD is about having lots of options. There is never only one right way to solve a problem; more options may exist. The more options you have, the better chance you will find a good solution to every problem. Requirements are those things which your system must do to work correctly. Object Oriented Analysis (OOA) is the primary technical step in developing a software system using the object oriented approach. OOA is based on a set of basic principles that include modeling the problem domain, representing behaviour and describing functions of the system. In Object-Oriented Design (OOD), the model created using OOA is transformed into a design model that works as a plan for software development. Object-oriented analysis and Design methods emerged in the 1980s, and object-oriented analysis methods emerged during the 1990s. In the early stage, object orientation was primarily associated with developing graphical user interfaces (GUIs), and a few other applications. In OOAD, a very basic concept followed is "one should model software systems as collections of cooperating objects, treating individual objects as instances of a class within a hierarchy of classes".

In this unit, we will discuss the basic notions of object orientation, including discussion on principles of object-orientation. You will learn the basic constructs of an object-oriented system. Also, we will discuss the basics of the object model with the

help of an example. Finally, in this unit, we will discuss the benefits of Object Orientation, and introduce to some OOAD tools.

1.1 OBJECTIVES

After going through this unit, you should be able to:

- describe basic concepts of object-oriented modeling,
- explain the principles of object orientation,
- explain various features of object orientations, including class and objects, links and Associations,
- explain the concept of Generalization and Inheritance,
- describe the benefits of OOM, and
- explain any given object models.

1.2 INTRODUCTION TO OBJECT ORIENTATION

The concept of OOAD was introduced during 1980s and early 1990s, and multiple OOAD approaches were suggested and the most popular were : Object Modelling Technique(OMT) for software modelling and designing. James Rumbaugh suggested it in 1991. He has suggested three types of models: Object, Dynamic and Functional Model for software modelling. Another approach was Booch methodology of Object Oriented Analysis and Design, which was proposed by Grady Booch proposed.

The OOAD approach is sequential in nature. The analysis phase establishes the system requirements and performs domain analysis from the customer perspective. Once the analysis phase is complete, the design phase (which is iterative in nature) maps the logic design to the physical design. The prototype is created and tested. It is one of the widely used approaches in Object Oriented Software Engineering. Another popular approach for Object-Oriented Software Engineering which was proposed by Ivar Jacobson 1992. It was the first object-oriented design method that made use of *use case diagram* in design. The phases included in his approach includes requirement gathering, analysis, design, implementation, and testing. He also introduced interaction diagrams to depict the sequence of operations in real-time and state transition diagrams to depict the change in state of object during execution. They are the pioneers in the field of object-oriented software engineering.

We know developing a model for a software system needs to follow well-defined steps. Prior to the system development or transformation, it is essential to have a blueprint for the system. It is similar to having a model/design for building a residential society. When we try to understand object-oriented modeling (OOM), we may find an entirely new way of thinking about problem analysis and creating a system model. The OOM methodology is about visualizing things using models organized around real-world concepts. Object-oriented models to help understand the problems, communicate with domain experts from a distance, model enterprises, and design the layout of computer programs and databases. Object-oriented models are represented using standard diagrams. A good model is always helpful in communication among project teams and assures architectural soundness.

The Object Modeling Technique (OMT) covers aspects of object-oriented analysis and design. OMT provides a very productive and practical way of Software development. The best part of object-oriented technology (OOT) is that it is not language-dependent. There is no need to consider a final implementation language during Object Oriented Modeling (OOM).

Analysis helps you ensure your system works in a real-world context. OOT combine structural, control and functional aspects of the system. We will discuss the structural, control and functional aspects in great detail in blocks 2 and block 3 of this course.

When we talk about a system, it means the complete application(project). It is important to note that the increasing complexity of systems increases the importance of modeling techniques. Because of its characteristics, OOM is a suitable modeling technique for handling complex systems. The OOM is basically used for building an application model, which includes the system's implementation details during the system's design.

Any system development follows a software development life cycle, which mainly includes: analysis, design, and implementation. During object-oriented modeling, identification and organization of application with respect to its domain is performed, rather than their final representation in any specific programming language. Once the system's modeling is complete, it can be implemented in any suitable programming language.

The OOM approach is an encouraging approach in which software developers have to think about the application domain throughout most of the software engineering life cycle. In this process, the developer is required to identify the inherent processes of the application. First, the developer organize, and understands the system correctly, and then they detail the data structure and functions of the system. Also, it is found that most good design comes from an analysis of bad designs. Therefore OOM analysis plays a very important role in proper system design. You can solve a big problem by breaking it into many functional pieces and then working on each piece individually. For this, you need to perform a proper analysis of the system. As "change is constant," your system should always improve every time you work on it. It is important for any designer/developer to make sure that their "software does what the customer wants to do".

Also, as an expert, you are expected to design the system that must work in the real world, not just when everything goes as you expect it to. When things in the system go wrong, your system must have alternate paths to reach the system's goal. Also, an important point to remember is that customers do not pay you for great code. They pay you for great software.

A requirement is usually a single thing, and you can test that thing to make sure that you have actually fulfilled the requirement. When looking into a system's architecture, it is the organizational structure of a system, including its decomposition into parts, their connectivity, interaction mechanism, and the guiding principles and decisions that you use to design a system. Good design will always reduce risk. Therefore good analysis and design of the system is a must for its successful implementation and customer satisfaction.

Object-Oriented Modeling and Design

It consists of the analysis and design phases, which are related but distinct. The first step of OOAD is identifying requirements and developing an object model of the application domain (OOA phase); software specifications are established during analysis. In this phase, using various models, an attempt is made to understand the problem by looking at real-world concepts.

The next step is OOD, in which the important construct used is the object. An object is the instance of a class and contains data structure in addition to its behavioural characteristics. Objects can interact with each other to design various applications and programs.

Object Oriented Modeling is divided into various stages:

The OOM of the system passes through the following steps:

- Requirement Analysis of System,
- Designing of system,
- Detailed designing with a focus on objects in the system, and
- Implementing the model.

Figure 1.1 shows the development process in object-oriented development model design.

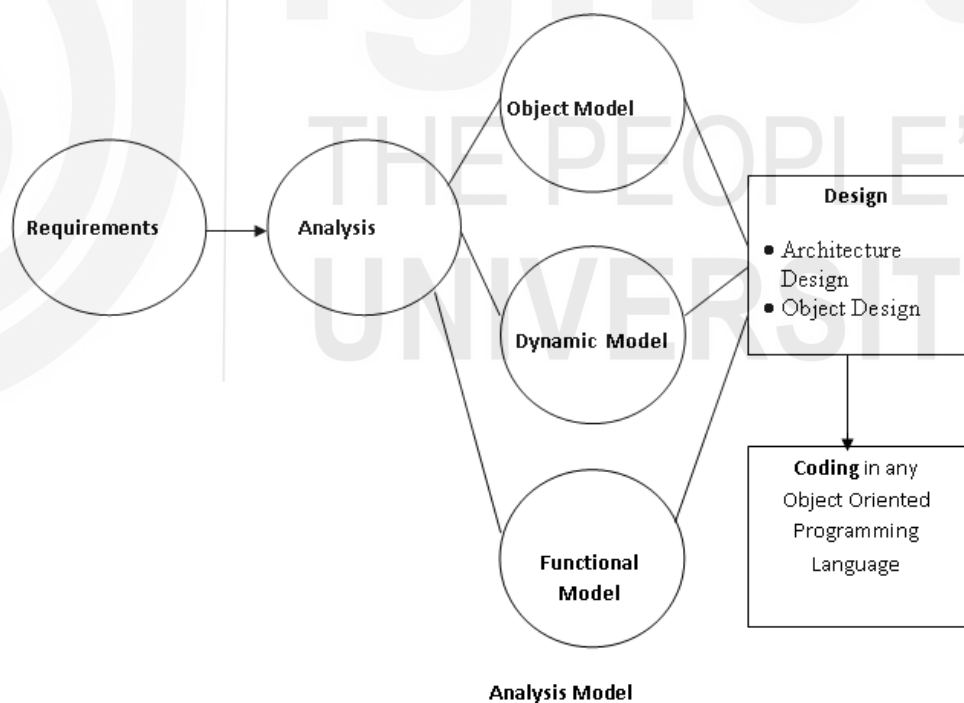


Figure 1.1: Object Oriented Development Model

System Analysis: At this stage, a problem statement is formulated, and the analyst builds a system model to encourage real-world situations. This phase shows the essential properties associated with the situation. The analysis model is a concise, precise abstraction and agreement on "how the desired system must be developed". You can say that here the objective is to provide a model that can be understood and also criticized by any application experts in the

area, no matter whether the expert is a programmer or not. All the system requirements and their properties should be defined at this phase. The main objective of this stage is to provide a model that can be understood by any naive user or expert in that area.

System Design: During this stage, the complete system architecture is designed. System design whole system is divided into subsystems based on both the system analysis model and the proposed architecture of the system. Based on the analysis model, the system's architecture is defined and designed in this phase. Also, when the system is big and complex, it is partitioned into various small subsystems based on the analysis model and system architecture.

Object Design: During this stage, a design model is developed based on the analysis model already developed in the earlier phase of development. The object design decides the data structures and algorithms needed to implement each of the classes in the system, using the implementation details given in the analysis model.

The steps followed in Object Designing are as follows:

- Procedures/operations performed on various classes are identified and finalized.
- The logic required to implement the procedure gets decided.
- All the paths for retrieving data are optimized.
- A list is prepared for all the permitted external interference.
- Classes and their associated operation are rearranged or adjusted to increase the inheritance properties between classes, and
- Packaging of all associated classes is done in a single module.

Implementation: At this stage, the final implementation of classes and relationships developed during object design takes place using any particular programming language. Implementation of database and hardware implementation (if needed) is done in this phase. The actual implementation must be done using software engineering practice. This helps to develop a flexible and extensible system.

Object Modeling Technique (OMT)

To cover the complete system description; whole object-oriented modeling is done using three kinds of models :

- Object model,
- Dynamic model, and
- Function model.

Object models are used to describe the objects and their relationships with each other in the system. The dynamic model describes the interaction among objects and information flow in the system. A functional model describes the data transformations in the system. **All three models are applicable during all the stages of development.** These models bear the responsibility of acquiring implementation details of the system development. It is important to note that you cannot describe a system completely until all three models are appropriately described. In block 2 of this course, we will discuss these three models in detail.

Before we further discuss object-oriented modeling, let us see how object-oriented development differs from structured development of the system. The structured approach is seen as the most direct way of implementing the desired goal. In the structured approach, the main emphasis is on specifying and decomposing system functionality. A structured approach has certain basic problems, such as if the requirements of the system change, then a system based on decomposing functionality may require massive restructuring, and the system gradually becomes unmanageable. In contrast to the structured approach, the basic focus of object-oriented approach is to **identify objects** from the application domain and then **associate procedures** (methods) around these identified objects.

Object-oriented development is an indirect way of system development because, in this approach, a holistic view of the **application domain is considered, and objects are identified** in the related problem domain. A historical view of the application helps in realizing the situations and characteristics of the system. Taking a **holistic view of the problem domain rather than considering the functional requirements of a single problem give an edge to object-oriented development**. Once the objects are created with the needed characteristics, they communicate with each other **by message passing** during problem-solving.

Object Modeling

Every object has some attributes and behaviour. Some common examples of objects include Car, Student, Banking, Book, Teacher, Employee etc. We can perform many functions on the object, including its categorization, description, organization of objects, combining and manipulating objects, etc. The object model can be represented graphically with the help of class diagrams and object diagrams. Object modeling will be discussed in Unit 6 of this course.

Dynamic Modeling

It is concerned with Time, States, and Transition from one state to another. Events that trigger the transition, Actions that happened after the event occurred and are instantaneous. The dynamic model will be discussed in Unit 7 of this course.

Functional Modeling

In functional modelling, the information is converted and processed. It represents how the data flow from one operation to another. The complete system is designed with the help of Data Flow Diagrams (DFDs). The DFD's main elements are Entities, Data Store, Processes or operations, file store, and Data Flow. Functional modelling will be discussed in Unit 8 of this course.

1.3 BASIC PHILOSOPHY OF OBJECT ORIENTATION

There are several characteristics of object-oriented technology. Now, let us discuss the essential characteristics of object-oriented systems.

Sharing of Structure and Behaviour

One of the reasons for the popularity of object-oriented techniques is that they encourage sharing at different levels. Inheritance is one of the main advantages of any object oriented language, because it gives scope to share basic code. Inheritance of both data structure and behaviour allows common structure (base class) to be used in designing many subclasses based on essential characteristics of the base class and develop new classes with less effort.

More broadly we can say that object-oriented development not only allows information sharing and reuse within an application, but also gives a base for project enhancement in future. As and when there is a need for adding new characteristics to the system, they can be added as an extension of existing basic features. This is done by using inheritance, and that too without major modification in the existing code. But be aware that just by using object orientation, you do not get a license to ensure reusability and enhancement. To ensure reusability and enhancement, you have to have a more general system design. This type of design can be developed if the system is properly studied and features of the proposed system are explored.

Emphasis on Object Structure, not on Operation Implementation

In object-orientation, the major emphasis is given to specifying the characteristics of the objects in a system rather than implementing these characteristics. The uses of an object depend highly on the facts of the application and regular changes during development. As requirements extend, the features supplied by an object are much more stable than the ways in which they are used. Hence software systems built on object structure are more secure.

While developing a system using the object-oriented approach, the main emphasis should be on the essential properties of the objects involved in the system rather than on the procedure structure used for implementation. During this process, “what an object is and its role in the system” is deeply thought about.

Using OOAD good software systems can be designed. In fact, OOAD provides a way to produce well-designed applications that satisfy both the customer and the programmer.

1.4 PRINCIPLES OF OBJECT ORIENTATION

In OOAD process, OO principles are used. These principles add flexibility, maintainability and extensibility to software. Also, applications of these principles in OOAD provide reusable design. In this section, we will discuss OO principles.

1.4.1 Abstraction

Abstraction is one of the very important concepts of object-oriented systems. Abstraction focuses on the essential, inherent aspects of an object of the system. It does not represent the accidental properties of the system. In system development, abstraction helps focus on what an object is supposed to do before deciding how it should be implemented. The use of abstraction protects the freedom to make decisions for as long as possible by avoiding intermediate commitments in problem-solving. Most modern languages provide data abstraction. With the abstraction, the ability to use inheritance and apply

polymorphism provides additional freedom and capability for system development. When you are using abstraction during analysis, you have to deal with application-domain concepts. You do not have to design and make implementation decisions at that point.

We can see abstraction in the real world all the time, and we just do not think of it in that specific way. One very common example you may take when you interact using your mobile phones. All you do is open an application and press a button to make a call. Actually, you are performing a simple task, but in reality a lot of things are happening that you may not know or do not need to know as a user. That is actually the abstraction, and it is applied as a principle in OOM.

1.4.2 Encapsulation

Encapsulation or information hiding is a principle by which we hide data that we don't want the user to see and show data that we want them to see. It is the feature of separating the external aspects of an object from the internal implementation details of that object. It helps in hiding the actual implementation of characteristics of objects. You can say encapsulation is hiding part of implementation that do internal things, and these hidden parts are not concerned with the outside world. Encapsulation enables you to combine data structure and behaviour in a single entity. Encapsulation helps in system enhancement. If there is a need to change the implementation of an object without affecting its external nature, encapsulation is of great help.

Encapsulation allows you to group your application into logical parts. You should always try to encapsulate what varies. Encapsulating what varies makes your application more flexible and easier to change. Any time you see duplicate code, look for a place to encapsulate it. Encapsulation is when you protect the information in your code from being used incorrectly. Also, encapsulation separates your data from the behaviour of your system.

1.4.3 Inheritance

Inheritance is taken in the sense of code reuse within object-oriented development. During modeling, we look at the resulting classes and try to group similar classes together so that code reuse can be enforced. Generalization, specialization, and inheritance have a very close association. Generalization is used to refer to the relationship among classes, and inheritance is used for sharing attributes and operations using the generalization relationship. In respect of inheritance, generalization and specialization are two phases of a coin in the sense that if a subclass is seen from a superclass, the subclass is seen as a specialized version of the superclass and in, reverse, a superclass looks like a general form of a subclass.

Use inheritance when one object behaves like another, rather than just when the IS-A relationship applies. During inheritance, a subclass may override a superclass feature by defining that feature with the same name. The overriding features (the subclass feature with the same names of superclass features) refines and replaces the overridden feature (the superclass feature).

Now let us look at the diagram given in figure1.2. In this diagram, Circle, Triangle, and Square classes are inherited from the Shape class. This is a case of single inheritance because here, one class inherits from only one class.

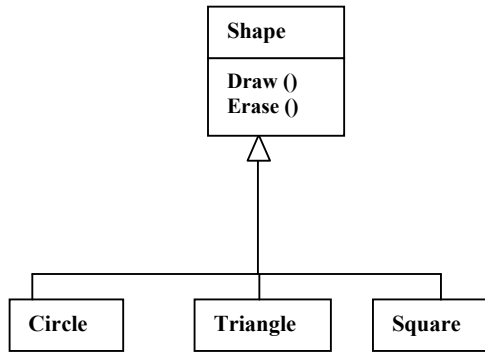


Figure 1.2: Single inheritance

Multiple inheritance is shown in figure 1.3. Here, one class is inherited from more than one class.

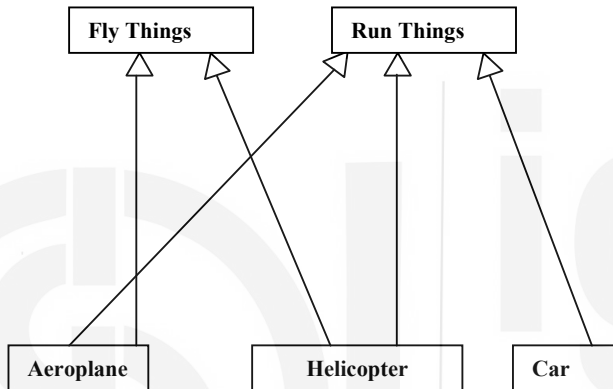


Figure 1.3: Multiple inheritance

1.4.4 Polymorphism

Polymorphism is closely related to inheritance. When one class inherits from another, then polymorphism allows a subclass to stand in for the superclass. The class hierarchy is the deciding factor in the case of more than one implementation of characteristics. An object-oriented program to calculate the area of different Figures would simply call the Find_Area operation on each figure, whether it is a circle, triangle, or something else. The decision of which procedure to use is made implicitly by each object, based on its class polymorphism makes maintenance more manageable because the calling code need not be modified when a new class is added. Polymorphism allows your applications to be more flexible and less resistant to change.



Check Your Progress- 1

- 1) What is Object-Oriented Modeling ?

.....

- 2) What are the different steps of the OOM process?

.....

- 3) Differentiate Object Oriented approach of system development from the structured approach of system development.

.....
.....
.....

1.5 BASIC CONSTRUCTS OF OBJECT ORIENTATION

In object-oriented modeling, objects and their characteristics are described. In any system, objects come into existence to play some role. In defining the roles of objects, some features of object orientation are used. In this section, we will discuss these features, which include:

- Class and Objects
- Links and Association
- Generalization and Inheritance

1.5.1 Class and Objects

A *class* is a collection of things or concepts that have the same characteristics. Each of these things or concepts is called an *object*.

In the next unit of this block, we will discuss that the class is the most fundamental construct within the UML.

Classes define the basic words of the **system being modeled**. Using a set of classes as the core vocabulary of a software project tends to greatly facilitate understanding and agreement about the meanings of terms and other characteristics of the objects in the system.

Classes can serve as the foundation for data modeling. Now, let us see how the characteristics that classes share are captured as attributes and operations. These terms are defined as follows:

- *Attributes* are named slots for data values that belong to the class. The different objects of a given class typically have at least some differences in the values of their attributes.
- *Operations* represent services that an object can request to affect the behaviour of the object or the system itself.

In units 2-5 of this block, we will cover the standard UML notation for OOM in detail. Here, we will mention about the standard notation of class. The notation for a class is **a box with three sections**. The top section contains the name of the class in boldface type, the middle section contains the attributes that belong to the class, and the bottom section contains the class's operations, as you can see in figure 1.4.

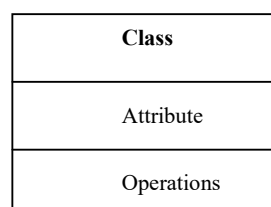


Figure 1.4: Class notation

You can also show a class without its attributes or its operations, or the name of the class can appear by itself, as shown in figure 1.5.

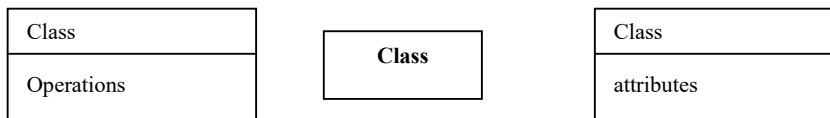


Figure 1.5: Alternate class notations

The naming convention for classes is as follow:

- Class names are simple nouns or noun phrases.
- Attribute names in a class are simple nouns or noun phrases. The first word is not capitalized, but subsequent words may be capitalised.
- Operation names are simple verbs. As with attributes, the first word is not capitalized, and subsequent words may be capitalised.

Objects

The notation for an object is the same in basic form as that for a class. There are three differences between the notations, which are:

- Within the top section of the class box, the name of the class to which the object belongs appears after a **colon**. The object may have a name, which appears before the colon, or it may be anonymous, in which case nothing appears before the colon.
- The contents of the top compartment are underlined for an object.
- Each attribute defined for the given class has a specific value for each object that belongs to that class.

You can see the notion of an object in figure 1.6.



Figure 1.6: Notation of object

If you look around, you will find many examples of real-world objects such as your books, desk, television, etc.

Everything that the software object knows (state) and can do (behaviour) is expressed by the variables and the methods within that object. In other words, all the objects share states and behaviour. Let us say that a software object that models your real-world bicycle would have variables indicating the bicycle's current state: its speed is 20 mph, its current gear is the 3rd gear, etc.

Communication by Message Passing

You will agree that a single object alone is generally not very useful. Objects usually appear as a component of a larger program or a system. Through the interaction of these objects, the functionality of systems is achieved. Software objects interact and communicate with each other by *message passing* to each other. When object X wants object Y to perform one of the methods of object Y, object X sends a message to object Y. Message passing provides two significant benefits:

- An object's characteristics are expressed through its methods, so message passing supports all possible interactions between objects.

- It closes the gap between objects. Objects do not need to be in the same process or even on the same machine to send and receive messages back and forth to each other.

Table 1.1: Difference between objects and classes

OBJECT	CLASS
Object is considered as a physical entity	Class is defined as an entity that exists logically
Any number of objects can be created for a class	Class is declared once.
Instance of a class is an object	Class is used to group similar objects

1.5.2 Links and Association

Links and associations are the basic means to establish relationships among objects and classes of the system. The next subsection will discuss links and associations used to represent relationships.

General Concepts

A link is a physical or conceptual connection between objects, for example, a student named Shrikant study in IGNOU. Mathematically, you can define a link as a tuple that is an ordered list of objects. Further, a link is also defined as an instance of an association. In other words you can say that an association is a group of links with a common structure and meanings, for example, a student studying in a university. All the links in an association connect objects from the same classes. A link is used to show a relationship between two (or more) objects.

Association and classes are similar as classes describe objects, and association describes **links**. Figure 1.7 shows us how we can show the association between Student and University

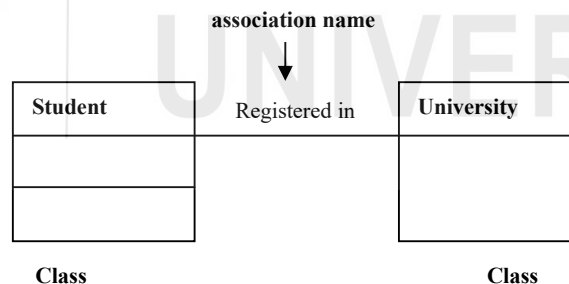


Figure 1.7: Association

Note that every association has roles. For example, in figure 1.8, two classes, Student and University, have their defined roles. Here you can also see that binary association has two roles, one from each class.

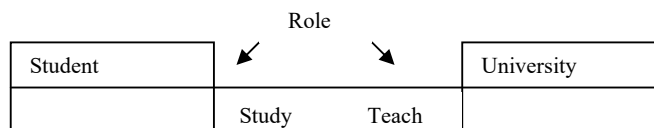


Figure 1.8: Roles in association

Associations may be binary, ternary, or have higher order. In exercise, the vast majority of association are binary or ternary associations. But a ternary association is formed compulsion; it cannot be converted into a binary

association. If a ternary association is decomposed into another association, some information will be lost. In figure 1.9 you can see a ternary association.

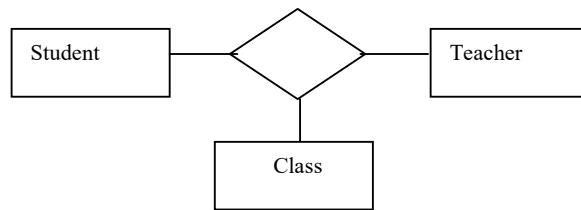


Figure 1.9: Ternary association

Multiplicity

Multiplicity in an association specifies how many objects participate in a relationship. Multiplicity decides the number of related objects. Multiplicity is generally explained as "one" or "many," but generally, it is a subset of the non-negative integers.

Table 1.2: Multiplicity Indicators

Indicator	Meaning
0..1	Zero or one
1	One only
0..*	Zero or more
1..*	One or more
n	Only n (where $n > 1$)
0..n	Zero to n (where $n > 1$)
1..n	One to n (where $n > 1$)

In associations, general movement is in both the directions of the relationships but if you want to be specific in any particular direction, you have to mark it with an arrow, as shown in figure 1.10.

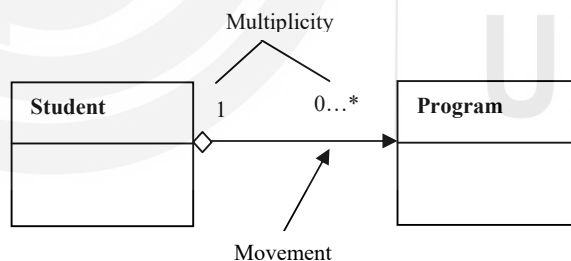


Figure 1.10: Association and movement

Aggregation

Aggregation is a special form of association, which models the "part-whole" or "a-part-of" or "has-a" relationship as an aggregate (the whole) and parts. The "is-a" relationship refers to inheritance and "has-a" relationship refers to Aggregation or Composition.

The most considerable property of aggregation is transitivity; if X is part of Y and Y is part of Z, then X is part of Z. Aggregation is seen as a relationship in which an assembly class is related to a component class. In this component, objects do not have separate existence and depend on composite objects; as you can see in figure 1.11, Exam Schedule does not have a separate existence.

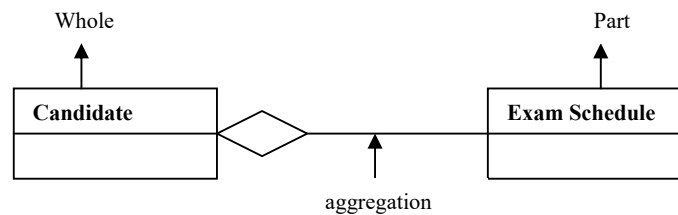


Figure 1.11: Association and whole-part relationship

1.5.3 Generalization and Inheritance

This section will discuss the concepts of generalization, inheritance, and their uses in OOM.

Generalization

Generalization and inheritance are powerful abstractions for sharing the structure and/or behaviour of one or more classes.

Generalization is the relationship between a class, and it defines a hierarchy of abstraction in which subclasses (one or more) inherit from one or more superclasses.

Generalization and inheritance are transitive across a subjective number of levels in the hierarchy. Generalization is an "is-a-kind of" relationship, for example, Saving Account is a kind of Account, PG student is a kind of Student, etc.

The notation for generalization is a **triangle connecting a superclass to its subclasses**. A line connects the superclass to the top of the triangle. Lines connect the subclasses to a horizontal bar attached to the base of the triangle. Generalization is a very useful construct for both abstract modeling and implementation. You can see in figure 1.12, a generalization of Account class.

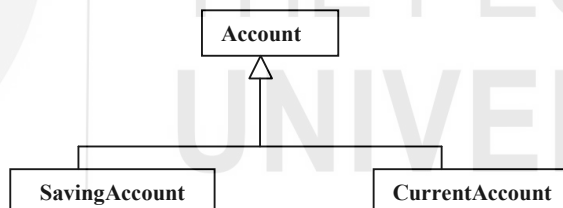


Figure 1.12: Generalization of account class

1.6 IDENTIFYING CLASSES

In object oriented modeling, system understanding and on the basis of that identification of classes is one of the key activity. Establishing relationship among different classes in the system are the first and foremost activity.

General practice to identify the classes in sysmen includes textual analysis. When you look for the nouns (and verbs) in your use case to figure out classes and related methods is called textual analysis. When you have a good use case, complete textual analysis is a quick and easy way to figure out the classes in your system. You have to just think about how the classes you do have can support the behaviour your use case describes. Nouns are candidate classes in the system. Knowledge of problem domain is essential to do proper textual analysis.

For finding the classes which could be representative of the problem domain and are useful in your problem solving.

Step -1

Find the nouns:

To find the nouns you have to read through the problem statement and the associated documentation and highlight the nouns. Nouns you identify in the text or conversation with the client generally translate into classes or objects, attributes, or attribute values. The verbs you identify in the analysis often reveal operations or associations. While interaction with the client/user you should ask questions related to the system with respect to the problem domain. Here it is expected that the client with whom you are interacting has a deep understanding of the problem domain.

Step -2

Evaluate the nouns to find classes

As an analyst you should ask questions to evaluate nouns to find classes:

1. Is the term identified actually needed within the scope of the project or not?
2. If needed, then does this noun qualify as a problem domain resource?
3. If answer to point 2 above is yes then think of an appropriate class name for the domain resource to which this noun is referring.
4. Otherwise, find whether this noun is an example of a class, for example, an object, an instance of a class?
5. Otherwise, decide whether the noun is an attribute of a class?
6. Otherwise, decide whether the noun is an example of an attribute.

Step- 3

Define the purpose

You need to have a proper description about why the class is a required resource of the problem domain.

1.6.1 An Object Model

Here, we have a simple object model of a University System with respect to different levels of courses offered by the University. As you can see in figure 1.13, which shows the basic classes of this system.

This diagram covers different levels of students in the hierarchy. Similarly, for other classes, such as Administration and Faculty, hierarchy level can be drawn to give a broader view of whole system.

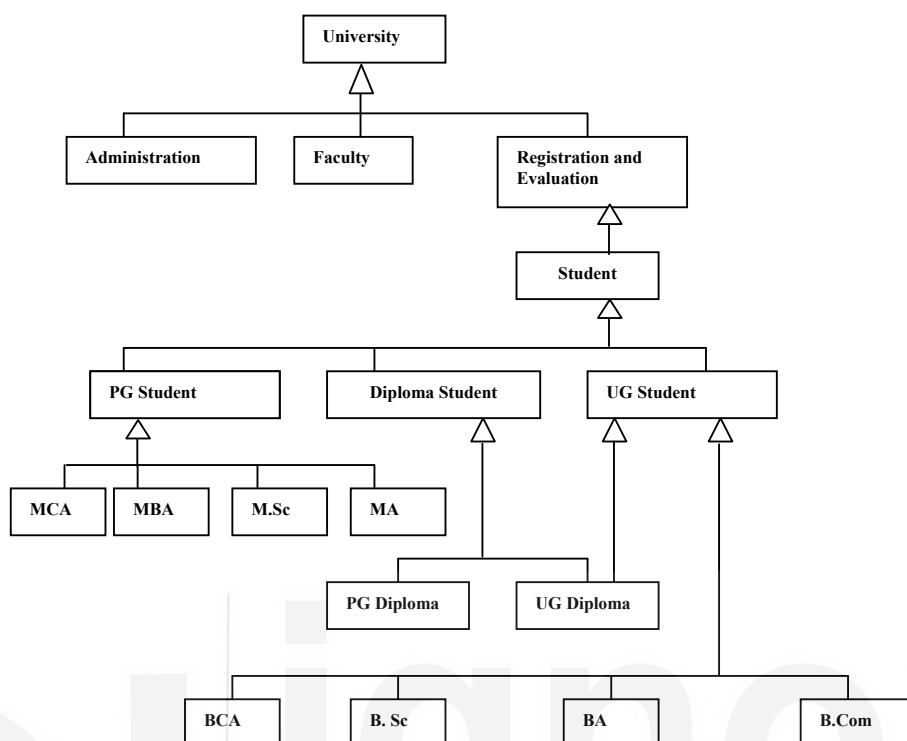


Figure 1.13: Object model for university system

1.7 BENEFITS OF OBJECT ORIENTATION (OO MODELING)

There are several advantages and benefits of object oriented modeling. Reuse and emphasis on quality are the major highlights of OOM. OOM provides resistance to change, encapsulation and abstraction, etc. Due to its very nature, all these features add to the systems development:

- Faster development
- Increased Quality
- Easier maintenance
- Reuse of software and designs, frameworks
- Reduced development risks for complex systems integration.

The conceptual structure of object orientation helps in providing an abstraction mechanisms for modeling, which includes:

- Classes
- Objects
- Inheritance
- Association

1.8 INTRODUCTION TO OBJECT ORIENTED ANALYSIS & DESIGN: TOOLS

Unified Modeling Language (UML) is a well-accepted language for OOAD. It is used for visualizing, specifying, constructing, and final documentation. The

basic building blocks of UML used for OOAD are things, relationships, and diagrams. Basically, the *Unified Modeling Language* focuses on the concepts of Booch, OMT, and Object Oriented Software Engineering (OOSE). The result of these concepts is a single, common, and widely usable modeling language for users of these and other methods. The *Unified Modeling Language* also promotes the concept of what can be done with existing methods.

Many modern applications are being developed based on object oriented principles such as classes, methods, and inheritance. For making and representing these features of the systems to be developed, some tools are used. These tools support UML features and building blocks. Object modeling CASE tools support object-oriented modeling notations and methodologies, and they also generate parts of object-oriented applications. Many of these object modeling CASE tools also support relational databases by performing arts of logic, and in some cases, physical database modeling and design, including schema generation and reverse engineering of RDBMS tables, and other elements. Many object oriented CASE tools are available to address new languages such as Java.

Here, we have tried to give a list collected from many sites and individual searches, for UML modeling tools. Since UML is a fast growing engineering , this list may keep on changing all the time. To draw UML diagrams, many tools are available. You may draw UML diagrams using simple drawing tools of MS-Words or Paint software. You may also draw UML diagrams using specialized software tools designed for this purpose. Here is the description of some tools/software which may be used for UML diagrams.

StarUML: (<https://staruml.io>)

It is an open-source software to develop a flexible, extensible, featureful, and freely-available UML platform. This platform is compatible with UML 2.x standard metamodel. You can draw different UML diagrams: Class, Object, Use Case, Component, Deployment, Composite Structure, Sequence, Communication, Statechart, Activity, Timing, Interaction Overflow, Information Flow and Profile Diagram using this tool. StarUML is a multi-lingual project, so any programming languages can be used to develop StarUML. For examples C/C++, Java, Visual Basic, Delphi, JScript, VBScript, C# and VB.NET etc. The key features of StarUML include the followings:

- Multi-platform support (MacOS, Windows and Linux)
- UML 2.x standard compliant
- Entity-Relationship Diagram (ERD)
- Data-flow diagram (DFD)
- Flowchart diagram
- Open APIs
- Various third-party extensions

BOUML(<https://www.bouml.fr>)

It is UMKL diagram designer software. It runs under Windows, Linux and MacOS X operating systems. It can be used for the automatic generation of code from the UML class diagrams and rebuild them from existing source code using the languages: C++, Java, PHP and MYSQL. It can also generate code

for Python and IDL. it runs under Linux, MacOS X and Windows. it is extensible, and the external tools

Visual Paradigm (<https://www.visual-paradigm.com>)

It is a UML CASE Tool supporting UML 2. It is a powerful visual modeling tool that helps you to build and manage your diagrams and model elements. It has a rich set of diagramming tools to create diagrams quickly and smoothly.

ArgoUML (<https://argouml.en.softonic.com>)

It is an open-source UML modeling tool which includes support for all standard UML diagrams, including statechart, activity diagrams, class diagrams, use-case diagrams, collaboration designs, deployment, and sequence diagrams, among others. It support a range of UML 1.4 diagrams, which includes: statechart, activity sketches, class diagrams, use-case diagrams, collaboration designs, deployment, and sequence diagrams, among others.

MagicDraw UML(<https://www.magicdraw.com>)

This tool has full support for all UML diagrams, it fully support UML 2.5.1, including class, use case, communication, sequence, state, activity, implementation, package, component, composite structure, deployment diagrams, and UML metamodel. MagicDraw's reverse engineering helps to get UML models from Java, C#, C++, CORBA IDL, EJB 2.0, DDL, CIL (MSIL), WSDL, and XML Schema source code.

Rational Rose

Rational Rose is an object-oriented Unified Modeling Language (UML) software design tool developed for visual modeling and component construction of enterprise-level software applications. ROSE = Rational Object Oriented Software Engineering

Visio Enterprise (https://microsoft.fandom.com/wiki/Microsoft_Visio)

Microsoft Visio is an enterprise class software. It contains a UML suite that can build UML diagrams within Visio. Currently Visio is part of the Office 365 suite.

☞ Check Your Progress -2

- 1) What is association? Give example of association.

.....
.....
.....

- 2) What is multiplicity in associations? Give example to explain multiplicity?

.....
.....
.....

- 3) What is inheritance? Give an example of multiple inheritance.

.....
.....
.....

1.9 SUMMARY

In this unit we have discussed the basic philosophy of object orientation, and the need for object oriented modeling. Also, object oriented principals are explained in this unit. This unit discussed, the concepts of abstraction, encapsulation, inheritance and polymorphism. These are the basic constructs of object orientation which makes it possible to develop systems in such a way that the system is flexible and open for reusability. Basic constructs of object orientation such as class, object, links and association, and generalization are explained in this unit. Also, we saw in this unit that how inheritance, generalization/specialization, and associations are represented. The reuse and quality of applications are mentioned as benefits of OOM. In the last section, some tools, which help in UML diagrams designs are mentioned.

1.10 SOLUTIONS/ANSWERS

Check Your Progress 1

- 1) Object oriented modeling is a approach through which modeling of the systems are done by visualizing the system based on the real world concepts using domain knowledge. Object oriented modeling is language independent and generally used for complex system development. Major benefits of object oriented modeling is that development of the system become fast, quality of the system is increase. It give freedom of use of existing design and code. Help in development of complex system with less risk due to the basic properties of object orientation which include class, objects and inheritance.
- 2) Steps involve in OOM are:
 - System Analysis
 - System Design
 - Object Design
 - Final implementation
- 3) Structured approach of problem solving is based on the concept of decomposition of system in to subsystem. In this approach of system development readjustment of some new changes in the system is very difficult. On the other hand in object oriented approach holistic view of application domain is considered and related object are identified. Further classification of objects are done. Object oriented approach give space for further enhancement of the system without too much increase in systems complexity.

Check Your Progress - 2

- 1) Association is used for establishing relationships between classes. Association describe links between (among) classes. For example, if a professor works in a university then it can be represented as shown in figure 1.4:

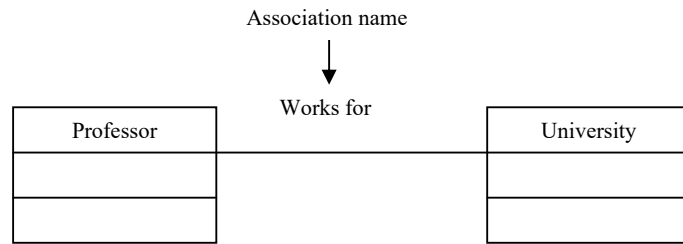


Figure 1.14: Association of professor and university

- 2) Multiplicity in an association indicate the number of objects participate in a relationship. For example in the association given in Figure 1.15 you can see that one player can play for one team at a time so here multiplicity is 1.

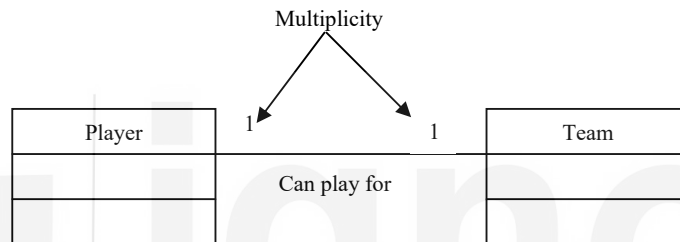


Figure 1.15: Multiplicity in association

- 3) Inheritance is an object orientation concept which allow reusability of design/code. Basic meaning of inheritance is that if one class is already defined than another class which also passes the property of existing class can be defined and inherit the property of existing class. For example, if a class named Student is defined and another class for Post Graduate students is to be defined then PG Student class can inherit Student class.

One example of multiple inheritance is a committee for students affair which consist of Faculty and Administrative staff member as shown in figure 1.16.

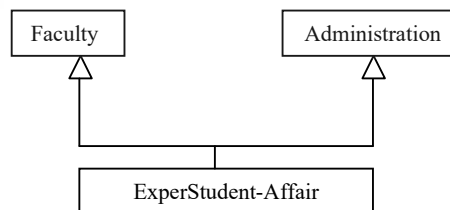


Figure 1.16: Multiple inheritance

1.11 FURTHER READINGS AND REFERENCES

- Grady Booch, James Rumbaugh and Ivar Jacobson, "The Unified Modeling Language User Guide", 2nd Edition, Addison-Wesley Object Technology Series, 2005.
- Brett D. McLaughlin, Gary Pollice, and David West, "Head First Object-Oriented Analysis & Design", O'Reilly, 2006.
- Grady Booch, Robert A. Maksimchuk, Michael W. Engle, Bobbi J. Young, Jim Conallen, Kelli A. Houston, "Object-Oriented Analysis and Design

with Applications," 3rd Edition, Addison-Wesley, 2007.

- James Rumbaugh, Ivar Jacobson, and Grady Booch, "Unified Modeling Language Reference Manual," 2nd Edition, Addison-Wesley Professional, 2004.
- John W. Satzinger, Robert B. Jackson, and Stephen D. Burd, "Object-oriented analysis and design with the Unified process," 1st Edition, Cengage Learning India, 2007.
- http://home.vinhuni.edu.vn/cuongvcc/wp-content/uploads/sites/109/2017/10/object_oriented_analysis_and_design_with_applications_2nd_edition_2788.pdf
- https://www.tutorialspoint.com/object_oriented_analysis_design/ooad_object_oriented_principles.htm
- https://www.ibm.com/docs/kk/SS5JSH_9.5.0/com.ibm.xtools.sample.rose.model.doc/topics/sample_rose_intro.html

